

# Segmented Network Security SIEM Lab

## Home Lab Security Project

Jon William Taylor

<https://github.com/JWT890/segmented-network-security-siem-lab>

Tools & Technology:

pfSense \* Wazuh \* Suricata \* ntopng \* Kali Linux \* Medusa \* Hydra \* Nmap \* Nikto \* Gobuster \* Metasploit  
\* Wireshark

## Contents

|                                        |    |
|----------------------------------------|----|
| Executive Summary .....                | 3  |
| Lab Architecture .....                 | 4  |
| 2.1: VM List .....                     | 4  |
| 2.2: VLAN Table .....                  | 4  |
| 2.3: Network Diagram.....              | 5  |
| Security Controls Implemented .....    | 6  |
| Firewall Rules .....                   | 6  |
| Intrusion Detection Suricata .....     | 6  |
| DNS and IP with pfBlockerNG .....      | 6  |
| SIEM Wazuh .....                       | 6  |
| VPN OpenVPN .....                      | 6  |
| Attack Simulation & Detection .....    | 7  |
| MITRE ATT&CK Mapping .....             | 7  |
| Key Findings .....                     | 9  |
| Lessons Learned .....                  | 10 |
| Technical Lessons .....                | 10 |
| Process Lessons .....                  | 10 |
| What to do differently next time ..... | 10 |
| Appendix .....                         | 11 |
| Firewall Rule Summary .....            | 11 |
| Wazuh Remote Syslog config .....       | 11 |
| Key Wazuh Rules Triggered .....        | 11 |
| Repo contents .....                    | 12 |

## Executive Summary

This project demonstrates the design and function of a segmented home lab network built in VirtualBox, with a pfSense firewall and 5 isolated VLANs, a Wazuh SIEM with endpoint agents and Sysmon, and a monitoring stack of Suricata, ntopng, and pfBlockerNG. This lab was used to simulate realistic attack scenarios using Kali Linux tools such as Medusa, Nmap, Nikto, Gobuster, and Metasploit, generating detectable events across both network and host layers that were logged with Wazuh and mapped to MITRE ATT&CK framework. Traffic was captured with Wireshark and analyzed alongside Suricata and Wazuh security events and timeline, demonstrating end-to-end visibility from attacker action through detection and looking. This was built from scratch, with assistance from Claude, to develop hands-on skills in firewall rule management, SIEM configuration, intrusion detection, threat hunting, and further knowledge in network segmentation.

## Lab Architecture

### 2.1: VM List

| VM Name          | OS                   | RAM  | Disk  | Role                     |
|------------------|----------------------|------|-------|--------------------------|
| pfSense          | FreeBSD 15 / pfSense | 2 GB | 15 GB | Firewall, VPN, DHCP, NAT |
| Linux-LabSwitch  | Ubuntu 22.04         | 4 GB | 20 GB | Bridge, VLAN switch      |
| Wazuh-SIEM       | Debian 12            | 8 GB | 80 GB | SIEM                     |
| servers-endpoint | Debian 12            | 2 GB | 20 GB | Monitored endpoint       |
| Lab-Attack       | Kali Linux 2026.2    | 4 GB | 50 GB | Attack platform          |

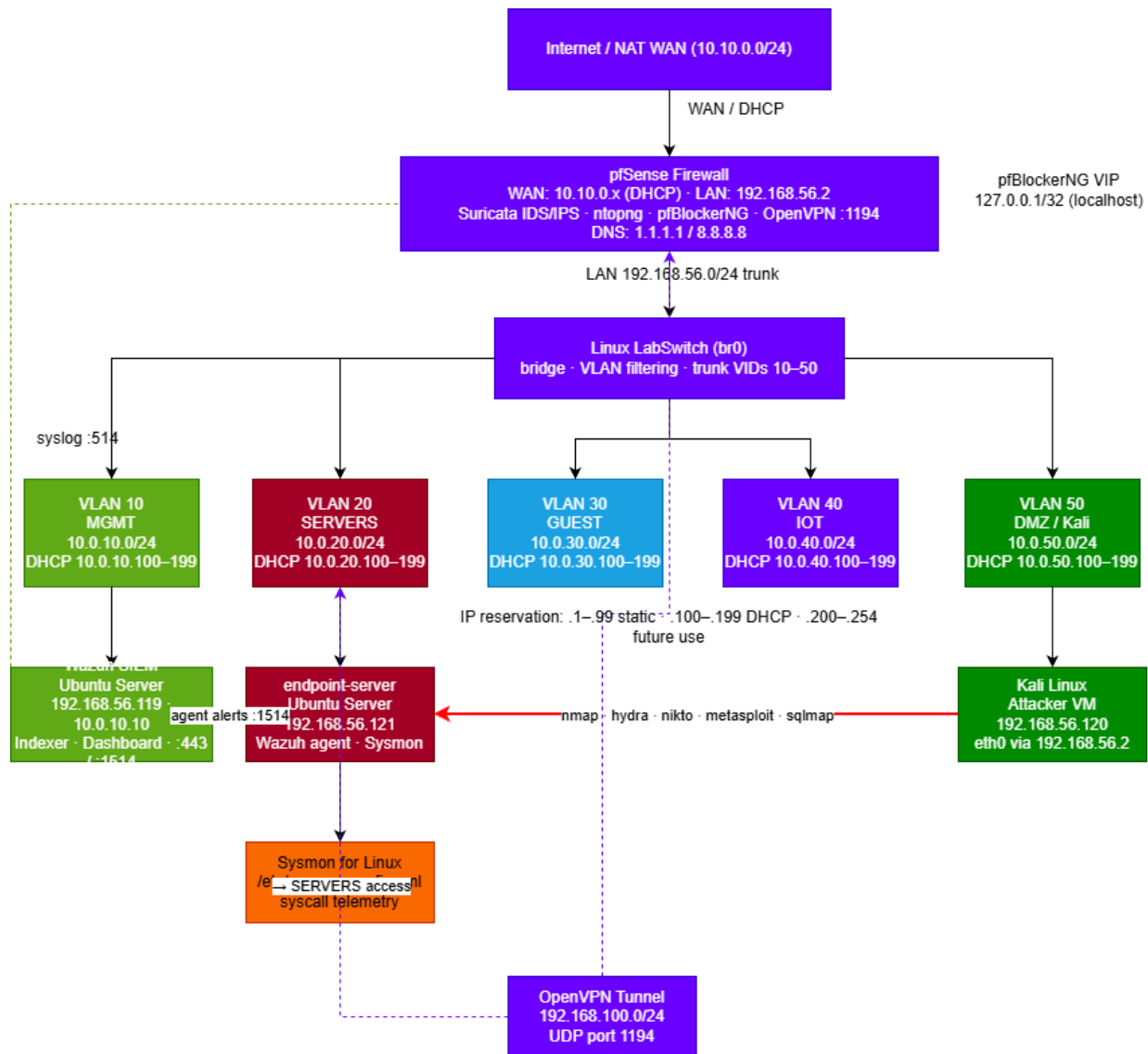
All the VMs in the table are running in VirtualBox on a host with 32 GB of RAM. Two virtual networks are used: lab-wan over NAT Network 10.10.0.0/24 for Internet and 192.168.56.0/24 runs on vboxnet0 or Virtualbox Host Adapter which connects the VMs together. pfSense acts as the central hub of the networks and handles the VLANs, NAT, and VPNs.

### 2.2: VLAN Table

| VLAN | Name    | Subnet       | Purpose                            | Policy                        |
|------|---------|--------------|------------------------------------|-------------------------------|
| 10   | MGMT    | 10.0.10.0/24 | Admin infra, Wazuh SIEM            | Full Access – admin only      |
| 20   | SERVERS | 10.0.20.0/24 | Monitored endpoint, Apache, agents | Outbound 80/443 only          |
| 30   | GUEST   | 10.0.30.0/24 | Simulated guest Wi-Fi              | Internet only, no LAN         |
| 40   | IOT     | 10.0.40.0/24 | IoT device simulation              | Fully isolated – no traffic   |
| 50   | DMZ     | 10.0.50.0/24 | Kali Linux                         | TCP 22/80/443 to SERVERS only |

The VLANs shown on the pfSense LAN NIC of .10 - .50 are subinterfaces. DHCP scopes are configured as per VLAN with the respective ranges with .100-.199 reserved for dynamic and .1-.99 reserved for static infrastructure assignments. Inter-VLAN routing is handled by pfSense with control over east-west traffic.

### 2.3: Network Diagram



### Legend

- VLAN 10 MGMT — admin-only access · Wazuh SIEM lives here
- VLAN 20 SERVERS — target endpoints · Wazuh agent · Sysmon
- VLAN 30 GUEST — internet-only · RFC1918 blocked
- VLAN 40 IOT — fully isolated · all traffic blocked
- VLAN 50 DMZ — Kali attacker · TCP 22/80/443 → SERVERS only
- Sysmon for Linux — syscall telemetry on endpoint

### Arrow types:

- Network / VLAN trunk flow
- Attack traffic (Kali → endpoint)
- Log / alert flow → Wazuh (syslog :514, agent :1514)
- OpenVPN tunnel (UDP 1194 · 192.168.100.0/24)

### IP Addressing Scheme (per VLAN):

- .1 = gateway (pfSense)
- .2–.99 = static assignments (servers, infrastructure)
- .100–.199 = DHCP pool (dynamic endpoints)
- .200–.254 = reserved for future use

# Security Controls Implemented

## Firewall Rules

pfSense uses a default-deny policy in which all traffic is blocked unless explicitly permitted. Rules are evaluated from the top down, with the first one considered. An RFC1918 alias groups all private address space 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) for reuse across multiple rules.

MGMT (VLAN 10): Full access – contains only administrative infrastructure

SERVERS (VLAN 20): Outbound TCP 80/443 only. All other traffic blocked and logged

GUEST (VLAN 30): Block to RFC1918 (logged) and allow to the internet only. No LAN access

IOT (VLAN 40): Block all traffic in all directions and fully air-gapped segment.

DMZ (VLAN 50): Allow TCP 22/80/443 and ICMP to SERVERS only. Block all else and logged

The DMZ rule set is asymmetric, Kali can initiate connections to SERVERS, but SERVERS cannot go back to DMZ, mirroring real-world attacker/defender network design.

## Intrusion Detection Suricata

Suricata runs on 3 interfaces with different modes. WAN operates in Legacy IDS mode (alert only), SERVERS (em1.20) operates in inline IPS mode, actively blocked matching signatures. DMZ (em1.50) operates in Legacy IDS mode, logging all Kali attack traffic for correlation. Rulesets enabled: Emerging Threats Open, Snort GPLv2 Community, Feodo Tracker Botnet C2, ABUSE.ch SSL Blacklist

## DNS and IP with pfBlockerNG

pfBlockerNG-devel was configured during initial setup and immediately blocked 17,257 known malicious IP addresses via Emerging Threats, Spamhouse DROP, and CINS Army Feeds, DNSBL was enabled with EasyList and EasyPrivacy to block ad and tracking domains at the DNS resolver level.

## SIEM Wazuh

Wazuh was deployed as an all-in-one install on the MGMT VLAN. pfSense forwards syslog (UDP 514) to the Wazuh manager, providing firewall event visibility. A Wazuh agent was deployed on the endpoint-server VM, with Sysmon for Linux providing process-level telemetry. The ossec.conf was configured to monitor /var/log/auth.log, /var/log/syslog, and Apache access/error logs.

## VPN OpenVPN

An OpenVPN server was configured on pfSense using SSL/TLS + User Auth. A Lab-CA certificate authority was created internally. The tunnel network is 192.168.100.0/24 was a split tunnel pushing only the SERVERS route (10.0.20.0/24) to connected clients. A keepalive directive was added to the client config to prevent NAT table expiry under VirtualBox.

## Attack Simulation & Detection

All attacks were run from Kali Linux at 192.168.56.120 against the endpoint-server at 192.168.56.121. Wireshark was run for a bit on the Kali eth0 interface for part of the attack phase when time was available, capturing a packet trace called lab\_attack\_capture.pcap, which I included in the repo.

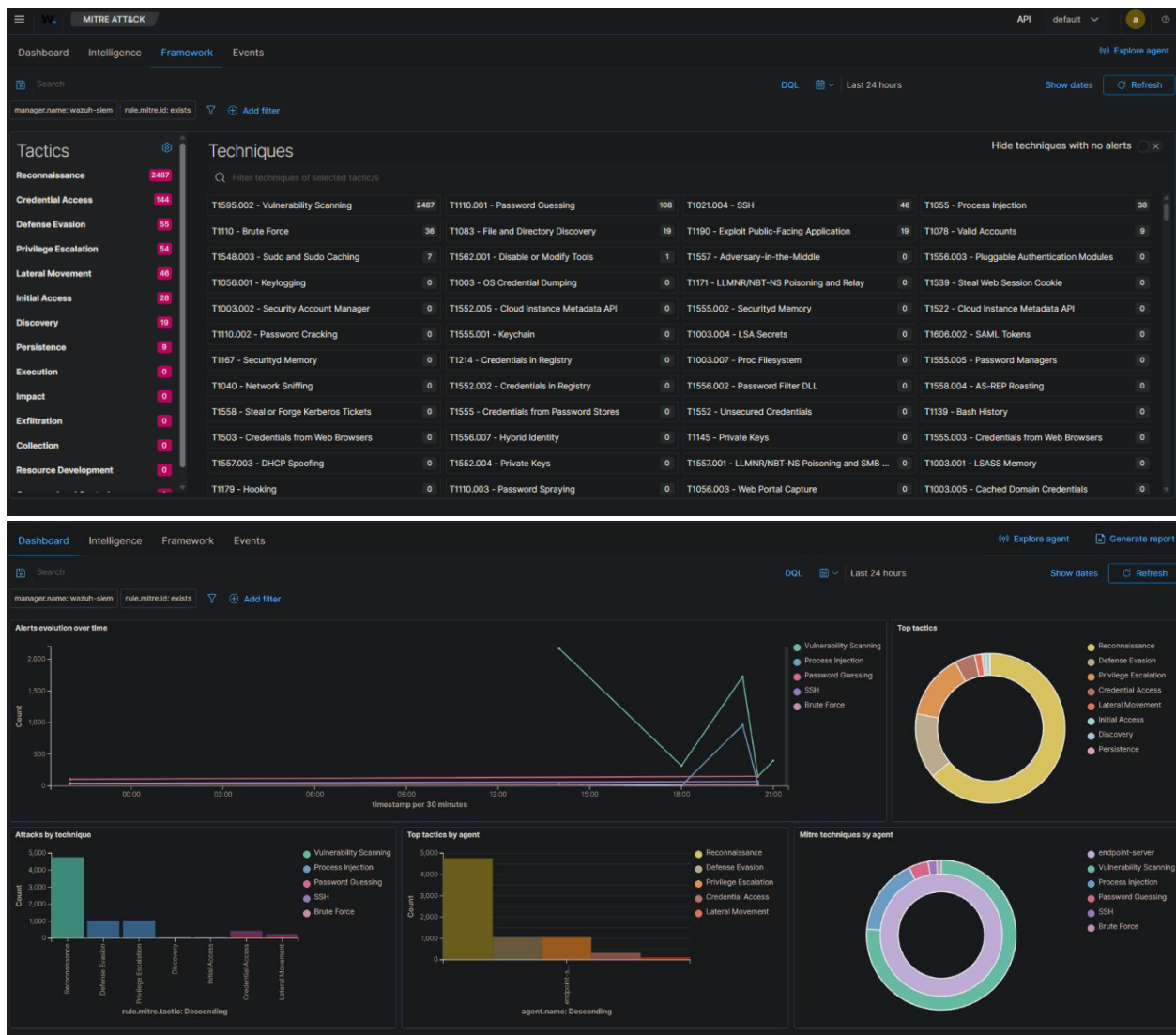
| Attack                 | Tool               | Target         | Detection         | Rule/Severity           |
|------------------------|--------------------|----------------|-------------------|-------------------------|
| SSH brute force        | Medusa             | 192.168.56.121 | Wazuh Host alert  | Rule 40111 – Level 10   |
| SSH brute force        | Hydra              | 192.168.56.121 | Wazuh host alert  | Rule 5712 – Level 10    |
| SSH user enum          | Metasploit         | 192.168.56.121 | Wazuh – rule 5710 | Rule 5710 – Level 5     |
| Web recon scan         | Nikto              | 192.168.56.121 | Wazuh web rule    | Rule 31101 – Level 5    |
| Directory brute force  | Gobuster           | 192.168.56.121 | Wazuh + MITRE Map | Rule 31151 – Level 10   |
| SQL injection          | SQLmap             | 192.168.56.121 | Wazuh web attack  | Rule 31103 – Level 6    |
| CVE vulnerability scan | nmap –script vuln  | 192.168.56.121 | CVEs identified   | CVE-2026-60002 CVSS 9.4 |
| ARP spoofing           | arp spoof          | 192.168.56.121 | Wireshark pcap    | Captured in pcap        |
| IDS signature trigger  | curl testmyids.com | WAN            | Suricata ET rule  | ET emerging threat      |
| Full port scan         | nmap -A -p-        | 192.168.56.121 | Suricata + Wazuh  | Rule 31151 – Level 10   |

From the Wireshark capture, the most significant detection was rule 40111 firing in Wazuh during Medusa and Hydra SSH brute-force attempts. After adding a function in the Wireshark pcap analyzer, with help from Claude, I was able to determine that the first packet it appeared in was packet number 356, according to result.txt from the output of the analyzer.

## MITRE ATT&CK Mapping

Wazuh automatically mapped detected events to the MITRE ATT&CK framework. The following were observed during the simulation

- T1595.002 – Active Scanning: Vulnerability Scanning (Gobuster, Nikto, nmap)
- T1110 – Brute Force: Password Guessing (Medusa, Hydra)
- T1055 – Process Injection (flagged during Apache install activity)
- T1078 – Valid Accounts: Credential Access attempts
- T1021.004 – Remote Services: SSH lateral movement attempts



The Wazuh MITRE dashboard above shows Recon as the dominant tactic with 4,500 events in the second image, followed by Defense Evasion and Privilege Escalation, each with 1,000 events. This shows the distribution between recon followed by target exploitation.



## Key Findings

| Finding                            | Severity          | Detail                                                                     |
|------------------------------------|-------------------|----------------------------------------------------------------------------|
| CVE-2026-60002 – OpenSSH 10.2pl    | CVSS 9.4 Critical | Identified via nmap vuln script on endpoint server port 22                 |
| CVE-2026-44691 – Apache 2.4.66     | CVSS 9.8 Critical | Multiple critical CVEs identified on web server                            |
| Missing HTTP security headers      | Medium            | Nikto found missing strict-transport-security, CSP, and permissions-policy |
| MITRE T1595.002 Active Scanning    | High              | Wazuh auto-mapped Gobuster recon to ATT&CK recon tactic                    |
| pfBlockerNG blocked 17,257 IPs     | Informational     | Threat intel feeds active from day one of deployment                       |
| SSH brute force unmitigated        | High              | No fail2ban or rate limiting – 40,000+ attempts reached the service        |
| ETag inode leakage – CVE-2003-1418 | Low               | Apache leaking inode data via ETag headers – easy to fix                   |

The most operationally significant finding was the SSH rate limiting on the endpoint. Medusa and Hydra were able to make thousands of authentication attempts without triggering an auto-block. Wazuh detected and alerted on the activity but made no automated response action. In a production environment, this would already be remediated, and this is for lab learning purposes.

## Lessons Learned

### Technical Lessons

For Wazuh ossec\_conf, make sure to delete the second <ossec\_config> since it caused wazuh-manager to fail silently when restarting, and always validate with xmllint —noout before restarting.

VirtualBox VLAN tagging does not work well on the guest NIC level. Kali couldn't get DHCP on VLAN 50 with the workaround being a static IP on the host only network and using explicit pfSense LAN firewall rules to control access.

Logging everything. Enabling all the pfSense syslog categories and forwarding to Wazuh generated a lot of noise, with a lot of rootcheck alerts in Wazuh events. Make sure to tune for alerts next time.

### Process Lessons

Make sure to document all errors with screenshots instead of saying if this happens, then do this instead of occasionally. One incident was getting rid of a firewall rule to move or edit in pfSense for Kali to connect and it taking out the LAN and figuring out how to get it back up. Especially in intermediate stages.

Run all the attacks with Wireshark running the whole time, missed a couple of the first ones since I did them before going to bed instead of blocking out a set amount of time for Wireshark to capture everything.

Take screenshots of intermediate stages like before for everything and not select times. Make sure to stop doing intensive parts right before going to bed.

### What to do differently next time

Add a second monitored endpoint on VLAN 30 (GUEST) from the start since some of the scenarios that had lateral movement were restrained some.

Give Wazuh a little boost in RAM from 8 GB to 10 GB since things loaded a little slower.

Configure fail2ban on the endpoint before doing the brute force attacks since it generates more interesting logs.

# Appendix

## Firewall Rule Summary

MGMT (OPT1): Allow any -> any

SERVERS (OPT2): Allow TCP OPT2net -> any port 80,443 (log)

Block any OPT2net -> RFC1918 (log)

GUEST (OPT3): Block any OPT3net -> RFC1918 (log)

Allow any OPT3net -> !RFC1918

IOT (OPT4): Block any OPT4net -> any (log)

DMZ (OPT5): Allow TCP OPT5net -> OPT2net port 22,80,443 (log)

Allow ICMP OPT5net -> OPT2net (log)

Block any OPT5net -> any (log)

## Wazuh Remote Syslog config

```
<remote>
  <connection>syslog</connection>
  <port>514</port>
  <protocol>udp</protocol>
  <allowed-ips>192.168.56.2</allowed-ips>
  <local_ip>192.168.56.119</local_ip>
</remote>
```

## Key Wazuh Rules Triggered

Rule 510 – Host based anomaly detection (rootcheck)

Rule 2501 – User authentication failure (PAM)

Rule 2502 – User authentication failure (PAM)

Rule 5402 – Successful sudo to ROOT executed

Rule 5557 – unix\_chkpwd: Password check failed

Rule 5710 – SSH login attempt with non-existent user

Rule 5712 – SSHD brute force attack

Rule 5758 – Maximum authentication attempts exceeded

Rule 5760 – sshd: authentication failed

Rule 5901 – New group added to the system

Rule 31101 – Web server 400 error code

Rule 31151 - Multiple web server 400 errors from same source IP

Rule 31516 – Suspicious URL access

Rule 40111 – Multiple authentication failures (Level 10)

Rule 52002 – AppArmor DENIED

## Repo contents

README.md – Full build step by step with screenshots

log / - Contains udp alerts from Wazuh

wirehark / - Packet analysis CSVs (attack\_classification.csv, extracted\_data.csv, extracted\_datav2.csv, ip\_counts.csv, lab\_attack\_capture.pcap, pcap\_analyzer2.py, port\_scan\_report.csv, and result.txt)

wazuh-dashboard-recovery.md – Wazuh dashboard troubleshooting notes

wazuh-dashboard-backend-timeout-resolution.md – Timeout fix documentation.